

Times: compile/linking/run

STRONG TYPING: $2 \neq 2.0 \rightarrow \text{float64}(2) == 2.0$

SCOPE: porzione di programma dove si può usare var

TIPI BASE

- INT: $_$, 8, 16, 32, 64 BIT $\rightarrow \%d$
- UINT: $==\text{int}$ ma senza segno
- FLOAT32/FLOAT64
- BOOL: true/false
- BYTE: alias uint8 $\rightarrow$ usato per rappresentare caratteri ascii
- RUNE: alias int32 $\rightarrow$ usato per caratteri UNICODE
- COMPLEX64/COMPLEX128 $\rightarrow$ immaginaria (es: $x=1+2i$) FUNC: real(x), imag(x)
- VAR GLOBALE $\rightarrow$ fuori da func $\rightarrow$ presa da tutte le func

TIPI COMPOSTI

- ARRAY: var A [N]int

- PUNTATORI:

```
var x int = 42
var p *int = &x // p ora contiene l'indirizzo di x
```

```
func raddoppia(n *int) {
    *n = *n * 2 // Usiamo * per accedere al valore all'indirizzo e modificarlo
}

func main() {
    x := 10
    raddoppia(&x) // Passiamo l'indirizzo di x usando &
    // Ora x vale 20!
}
```

controllare sempre se un puntatore è nil prima di usarlo: *if p != nil { ... }*

- *SLICE: array senza dimensione, deciso a runtime: `var S []int S=make([]int, N)`
- *MAP: `var m map[string]int m=make(map[string]int)`
- STRUCT:

```
type Libro struct {
    Titolo string
    Autore string
    Pagine int
    Pronto bool
}
```

```
func main() {
    // Inizializzazione di una struct
    mioLibro := Libro{
        Titolo: "Programmazione in Go",
        Autore: "A. Ceselli",
        Pagine: 250,
        Pronto: false,
    }
}
```

```
mioLibro.Titolo, mioLibro.Autore)
```

Speciali:

- APPEND: mette in coda il valore → `append(S, 100)`
- `S=make([]int, 3, 5)` → mette anche capacità massima
- `Delete(nome_map, chiave)` → rimuove la chiave e il relativo valore

TYPE

- Usato in struct
- `type nome int32` → crea nuovo tipo distinto (es: `type x float64 != y=float64`)
- `type nome=int32` → alias per int32

SWITCH:

```
fmt.Scan(&x)
switch x {
case "luca":
    fmt.Println("è", x)
case "marco":
    fmt.Println("è", x)
default:
    fmt.Println("exit")
}
```

RANGE: restituisce indice e copia valore

```
numeri := []int{10, 20, 30}
for i, v := range numeri {
    fmt.Printf("Posizione %d: valore %d\n", i, v)
}
```

OPERATORI INTRABIT:

```
// 2. Operazione Intrabit (Bitwise)
// Per passare da minuscolo a maiuscolo in ASCII, bisogna "spegnere" il sesto bit.
// Usiamo una maschera: 11011111 (che in decimale è 223 o 0xDF)
var maschera byte = 0xDF // 11011111
maiuscola := minuscola & maschera

fmt.Println("--- Dopo operazione Bitwise AND ---")
fmt.Printf("Nuovo valore binario: %08b\n", maiuscola) // 01000001
fmt.Printf("Nuovo carattere: %c\n", maiuscola)         // 'A'
fmt.Printf("Nuovo codice ASCII: %d\n\n", maiuscola)    // 65

// 3. Esempio di Shift
// Moltiplichiamo il valore per 2 usando lo shift a sinistra
valore := byte(10)
risultatoShift := valore << 1
fmt.Println("--- Esempio Shift ---")
fmt.Printf("Partenza: %d (%08b)\n", valore, valore)
fmt.Printf("Shift << 1 (10*2): %d (%08b)\n", risultatoShift, risultatoShift)
```

METODO *func (ricevitore struct) nome_func(input)(output)* [struct]

- ricevitore di valore → deve avere nome con nome struct (es: struct=luca, *func* (x luca) nome...) → crea copia, se si cambia funzione dentro, cambia solo copia
- ricevitore di puntatore → deve avere nome con nome struct (es: struct=luca, *func* (x *luca) nome...) → no copia, cambia originale

METODO *func (ricevitore) nome_func(input)(output)* [non-struct]

- solo su tipi dichiarati, NON INT, FLOAT... originali

FUNCTION CALL FRAME

creato e attivato function call frame ogni volta che viene eseguito il programma → spazio allocato a memoria dinamica (AUTOMATIC ALLOCATION) → disattivato e rimosso a fine programma

CONTIENE: I/O, variabili locali e passaggio per copia (passaggio parametri da main a func)

SCOPE: porzione di programma dove si può usare var

TABELLA % (con numero davanti (%8b) riserva 8 spazi per il numero (fmt.Printf("...", ...) / fmt.Scanf(&...), ...)

Verbo	Descrizione	Esempio / Note
<code>%v</code>	Valore predefinito (default format). È il "jolly": Go sceglie il formato più naturale per il dato.	Molto utile per debugging rapido.
<code>%T</code>	Stampa il Tipo della variabile.	Es: <code>int</code> , <code>string</code> , <code>float64</code> .
<code>%d</code>	Numero intero in base 10 (decimale).	Usato per i classici conteggi.
<code>%b</code>	Numero intero in base 2 (binario).	Utile per vedere i bit (es. <code>%08b</code>).
<code>%x</code> / <code>%X</code>	Numero intero in base 16 (esadecimale).	<code>%x</code> lettere minuscole, <code>%X</code> maiuscole.
<code>%c</code>	Il carattere (rune) corrispondente.	Trasforma un codice numerico nel suo simbolo (es. <code>65</code> → <code>A</code>).
<code>%f</code>	Numero in virgola mobile (float).	Es: <code>3.141592</code> .
<code>%s</code>	Una stringa di testo o una slice di byte.	Stampa il contenuto testuale.
<code>%t</code>	Valore booleano .	Stampa <code>true</code> o <code>false</code> .
<code>%%</code>	Stampa il simbolo % letterale.	Necessario perché <code>%</code> da solo indica l'inizio di un verbo.

MATH

Categoria	Funzione	Descrizione	Esempio
Potenze e Radici	<code>Sqrt(x)</code>	Calcola la radice quadrata di x .	<code>math.Sqrt(16)</code> → 4.0
	<code>Pow(x, y)</code>	Eleva x alla potenza di y (x^y).	<code>math.Pow(2, 3)</code> → 8.0
	<code>Cbrt(x)</code>	Calcola la radice cubica di x .	<code>math.Cbrt(27)</code> → 3.0
Arrotondamenti	<code>Ceil(x)</code>	Arrotonda per eccesso all'intero più vicino.	<code>math.Ceil(3.1)</code> → 4.0
	<code>Floor(x)</code>	Arrotonda per difetto all'intero più vicino.	<code>math.Floor(3.9)</code> → 3.0
	<code>Round(x)</code>	Arrotonda all'intero più vicino (.5 va all'intero lontano da zero).	<code>math.Round(3.5)</code> → 4.0
Valori Assoluti e Segno	<code>Abs(x)</code>	Restituisce il valore assoluto di x .	<code>math.Abs(-5.5)</code> → 5.5
	<code>Max(x, y)</code>	Restituisce il valore maggiore tra x e y .	<code>math.Max(10, 20)</code> → 20.0
	<code>Min(x, y)</code>	Restituisce il valore minore tra x e y .	<code>math.Min(10, 20)</code> → 10.0
Trigonometria	<code>Sin(x)</code> , <code>Cos(x)</code> , <code>Tan(x)</code>	Seno, coseno e tangente (in radianti).	<code>math.Sin(math.Pi / 2)</code> → 1.0
Esponenziali e Logaritmi	<code>Log(x)</code>	Logaritmo naturale (base e) di x .	<code>math.Log(math.E)</code> → 1.0
	<code>Log10(x)</code>	Logaritmo in base 10 di x .	<code>math.Log10(100)</code> → 2.0
	<code>Exp(x)</code>	Calcola e^x .	<code>math.Exp(1)</code> → 2.718...

MATH/CMPLX

Funzione	Descrizione	Note Matematiche
<code>cmplx.Abs(z)</code>	Calcola il Modulo (o valore assoluto).	$\sqrt{re^2 + im^2}$
<code>cmplx.Phase(z)</code>	Calcola l' Argomento (fase).	L'angolo θ nel piano complesso.
<code>cmplx.Sqrt(z)</code>	Radice Quadrata.	Funziona anche con numeri negativi (es. $\sqrt{-1} = i$).
<code>cmplx.Pow(x, y)</code>	Elevamento a Potenza.	x^y dove sia x che y possono essere complessi.
<code>cmplx.Exp(z)</code>	Esponenziale.	Calcola e^z .
<code>cmplx.Log(z)</code>	Logaritmo Naturale.	Il logaritmo di base e di z .
<code>cmplx.Conj(z)</code>	Coniugato.	Inverte il segno della parte immaginaria ($a + bi \rightarrow a - bi$).
<code>cmplx.Sin / Cos / Tan</code>	Funzioni Trig.	Funzioni trigonometriche applicate ai complessi.

MATH/RAND `rand.Seed(time.Now().UnixNano())`

`n := rand.Intn(max - min + 1) + min` //numero rand tra min e max

Funzione	Cosa restituisce	Range / Note
<code>rand.Intn(n)</code>	Un intero <code>int</code>	Da 0 a $n - 1$. È la più usata in assoluto.
<code>rand.Float64()</code>	Un numero <code>float64</code>	Da 0.0 a 1.0 (escluso).
<code>rand.Int()</code>	Un intero positivo	Un valore casuale non negativo nel range del tipo <code>int</code> .
<code>rand.Perm(n)</code>	Una slice di interi <code>[]int</code>	Una permutazione casuale dei numeri da 0 a $n - 1$.
<code>rand.Seed(value)</code>	Nulla (imposta il seme)	Fondamentale: determina la sequenza dei numeri.

OS

Categoria	Funzione	Descrizione
File System	<code>os.Create(name)</code>	Crea o tronca un file. Se esiste già, lo svuota.
	<code>os.Open(name)</code>	Apre un file in sola lettura.
	<code>os.OpenFile(...)</code>	Apre un file con permessi specifici (lettura, scrittura, append).
	<code>os.Remove(name)</code>	Elimina un file o una directory vuota.
	<code>os.Rename(old, new)</code>	Rinomina o sposta un file/directory.
Directory	<code>os.Mkdir(name, perm)</code>	Crea una nuova directory con i permessi specificati.
	<code>os.Getwd()</code>	Restituisce la "Current Working Directory" (cartella corrente).
	<code>os.Chdir(dir)</code>	Cambia la directory corrente del processo.
Informazioni	<code>os.Stat(name)</code>	Restituisce i metadati (dimensione, permessi, data modifica).
	<code>os.IsNotExist(err)</code>	Verifica se l'errore ricevuto indica che il file non esiste.
Processo	<code>os.Exit(code)</code>	Interrompe il programma immediatamente con un codice di stato.
	<code>os.Args</code>	(Variabile) Slice che contiene gli argomenti passati da riga di comando.
Ambiente	<code>os.Getenv(key)</code>	Recupera il valore di una variabile d'ambiente (es. PATH).
	<code>os.Setenv(key, val)</code>	Imposta il valore di una variabile d'ambiente.

STRCONV (da string a numero)

Funzione	Descrizione	Esempio
<code>strconv.Atoi(s)</code>	"ASCII to Integer". La più comune per convertire stringhe in <code>int</code> .	<code>n, _ := strconv.Atoi("42")</code>
<code>strconv.ParseInt(s, base, bitSize)</code>	Converte stringhe in interi specificando la base (es. 2, 10, 16).	<code>i, _ := strconv.ParseInt("101", 2, 64)</code>
<code>strconv.ParseFloat(s, bitSize)</code>	Converte una stringa in un numero decimale (<code>float64</code>).	<code>f, _ := strconv.ParseFloat("3.14", 64)</code>
<code>strconv.ParseBool(s)</code>	Accetta "t", "T", "true", "0", "f", "F", "false".	<code>b, _ := strconv.ParseBool("true")</code>

STRCONV (da numero a stringa)

Funzione	Descrizione	Esempio
<code>strconv.Itoa(i)</code>	"Integer to ASCII". Converte un <code>int</code> in stringa.	<code>s := strconv.Itoa(42) // "42"</code>
<code>strconv.FormatInt(i, base)</code>	Converte un intero in una stringa con base specifica (es. binario).	<code>s := strconv.FormatInt(255, 16) // "ff"</code>
<code>strconv.FormatFloat(f, fmt, prec, bitSize)</code>	Converte un float con controllo su decimali e formato.	<code>s := strconv.FormatFloat(3.1415, 'f', 2, 64) // "3.14"</code>
<code>strconv.FormatBool(b)</code>	Converte un booleano nella stringa "true" o "false".	<code>s := strconv.FormatBool(true)</code>

STRCONV (quoting)

Funzione	Descrizione	Esempio
<code>strconv.Quote(s)</code>	Aggiunge le doppie virgolette e gestisce i caratteri di escape.	<code>s := strconv.Quote("Ciao\n") // "Ciao\n"</code>
<code>strconv.Unquote(s)</code>	Rimuove le virgolette da una stringa formattata.	<code>s, _ := strconv.Unquote(`\"Ciao\"`)</code>

BUFIO (scanner) (ctrl-D)

Funzione / Metodo	Descrizione	Esempio di utilizzo
<code>bufio.NewScanner(r)</code>	Crea un nuovo scanner che legge da un <code>io.Reader</code> (come un file o lo standard input).	<code>sc := bufio.NewScanner(os.Stdin)</code>
<code>sc.Scan()</code>	Legge il prossimo "token" (di default la riga successiva). Restituisce <code>false</code> a fine input.	<code>for sc.Scan() { ... }</code>
<code>sc.Text()</code>	Restituisce il contenuto appena letto sotto forma di <code>stringa</code> .	<code>riga := sc.Text()</code>
<code>sc.Bytes()</code>	Restituisce il contenuto appena letto sotto forma di <code>slice di byte</code> (<code>[]byte</code>).	<code>dati := sc.Bytes()</code>
<code>sc.Split(splitFunc)</code>	Cambia il modo in cui lo scanner divide l'input (es. per parole invece che per righe).	<code>sc.Split(bufio.ScanWords)</code>

```
scanner := bufio.NewScanner(os.Stdin)

fmt.Println("Inserisci del testo (Premi Ctrl+D per terminare):")

// Il ciclo continua finché c'è input e si ferma con Ctrl+D
for scanner.Scan() {
    linea := scanner.Text()
    fmt.Printf("Hai scritto: %s\n", linea)
}

// Dopo il ciclo, è buona norma controllare se è uscito per EOF o per un
if err := scanner.Err(); err != nil {
    fmt.Fprintln(os.Stderr, "Errore durante la lettura:", err)
}

func main() {
    // 1. Controllo se è stato passato l'argomento (os.Args[0] è il nome del programma)
    if len(os.Args) < 2 {
        fmt.Println("Errore: specifica il nome del file. Esempio: go run main.go miofile.txt")
        return
    }

    // 2. Prendo il nome del file dagli argomenti
    nomeFile := os.Args[1]

    // 3. Apro il file fisico
    file, err := os.Open(nomeFile)
    if err != nil {
        fmt.Printf("Errore nell'apertura del file %s: %v\n", nomeFile, err)
        return
    }
    // Fondamentale: chiudere il file alla fine
    defer file.Close()

    // 4. Inizializzo lo scanner sul file appena aperto
    scanner := bufio.NewScanner(file)

    fmt.Printf("--- Inizio lettura file: %s ---\n", nomeFile)

    for scanner.Scan() {
        // Stampo ogni riga del file
        fmt.Println(scanner.Text())
    }

    // 5. Controllo eventuali errori di scansione (es. file corrotto durante lettura)
    if err := scanner.Err(); err != nil {
        fmt.Printf("Errore durante la scansione: %v\n", err)
    }
}
```

BUFIO (reader)

Funzione / Metodo	Descrizione	Note
<code>bufio.NewReader(r)</code>	Crea un lettore con un buffer predefinito (solitamente 4KB).	Utile per file grandi.
<code>r.ReadString(delim)</code>	Legge fino alla prima occorrenza del delimitatore scelto.	Es: <code>r.ReadString('\n')</code>
<code>r.ReadByte()</code>	Legge un singolo byte dall'input.	Ritorna <code>(byte, error)</code> .
<code>r.Peek(n)</code>	Permette di "sbirciare" i prossimi <code>n</code> byte senza avanzare la posizione di lettura.	Utile per parsing complessi.

BUFIO (writer)

Funzione / Metodo	Descrizione	Importanza
<code>bufio.NewWriter(w)</code>	Crea un nuovo scrittore bufferizzato.	Riduce il numero di chiamate di sistema.
<code>w.WriteString(s)</code>	Scrive una stringa nel buffer.	Non scrive ancora sul disco/video.
<code>w.Flush()</code>	Fondamentale: invia tutti i dati accumulati nel buffer alla destinazione finale.	Se dimenticato, i dati potrebbero andare persi.

UNICODE (test)

<code>unicode.IsDigit(r)</code>	Verifica se la rune è un numero (0-9).	<code>unicode.IsDigit('5') → true</code>
<code>unicode.IsLetter(r)</code>	Verifica se la rune è una lettera (di qualsiasi alfabeto).	<code>unicode.IsLetter('A') → true</code>
<code>unicode.IsLower(r)</code>	Verifica se la rune è una lettera minuscola.	<code>unicode.IsLower('g') → true</code>
<code>unicode.IsUpper(r)</code>	Verifica se la rune è una lettera maiuscola.	<code>unicode.IsUpper('G') → true</code>
<code>unicode.IsSpace(r)</code>	Verifica se è uno spazio bianco (spazio, tab, invio).	<code>unicode.IsSpace('\n') → true</code>
<code>unicode.IsPunct(r)</code>	Verifica se è un carattere di punteggiatura.	<code>unicode.IsPunct('!') → true</code>
<code>unicode.IsSymbol(r)</code>	Verifica se è un simbolo matematico o monetario.	<code>unicode.IsSymbol('€') → true</code>

UNICODE (conversione)

Funzione	Descrizione	Esempio
<code>unicode.ToLower(r)</code>	Converte la rune in minuscolo.	<code>unicode.ToLower('A') → 'a'</code>
<code>unicode.ToUpper(r)</code>	Converte la rune in maiuscolo.	<code>unicode.ToUpper('a') → 'A'</code>
<code>unicode.ToTitle(r)</code>	Converte la rune in formato "Title Case".	Spesso coincide con ToUpper.

UNICODE (range e tabelle)

Funzione	Descrizione	Esempio
<code>unicode.Is(tab, r)</code>	Verifica se una rune appartiene a una specifica tabella.	<code>unicode.Is(unicode.Latin, 'a')</code>
<code>unicode.In(r, ranges)</code>	Verifica se la rune appartiene a uno dei range indicati.	Controllo su più alfabeti.

STRINGS

Ricerca	<code>Contains(s, sub)</code>	Restituisce <code>true</code> se la sottostringa è presente.	<code>strings.Contains("Golang", "go") // false</code>
	<code>HasPrefix(s, pre)</code>	Verifica se la stringa inizia con un certo prefisso.	<code>strings.HasPrefix("file.txt", "file")</code>
	<code>Index(s, sub)</code>	Restituisce la posizione (indice) della prima occorrenza.	<code>strings.Index("mela", "e") // 1</code>
Modifica	<code>ToLower(s) / ToUpper(s)</code>	Converte tutto in minuscolo o maiuscolo.	<code>strings.ToUpper("ciao") // "CIAO"</code>
	<code>ReplaceAll(s, old, new)</code>	Sostituisce tutte le occorrenze di una sottostringa.	<code>strings.ReplaceAll("cane gatto", "cane", "fox")</code>
	<code>TrimSpace(s)</code>	Rimuove spazi bianchi e invio all'inizio e alla fine.	<code>strings.TrimSpace(" test \n") // "test"</code>
Suddivisione	<code>Split(s, sep)</code>	Taglia la stringa in una slice usando un separatore.	<code>strings.Split("a,b,c", ",") // ["a" "b" "c"]</code>
	<code>Join(slice, sep)</code>	Unisce gli elementi di una slice in una stringa sola.	<code>strings.Join([]string{"A", "B"}, "-") // "A-B"</code>
Conteggio	<code>Count(s, sub)</code>	Conta quante volte compare una sottostringa.	<code>strings.Count("banana", "a") // 3</code>

SORT

Tipi Base	<code>Ints(s)</code>	Ordina una slice di interi in ordine crescente.	<code>sort.Ints([]int{3, 1, 2})</code>
	<code>Strings(s)</code>	Ordina una slice di stringhe in ordine alfabetico.	<code>sort.Strings([]string{"b", "a"})</code>
	<code>Float64s(s)</code>	Ordina una slice di float64 in ordine crescente.	<code>sort.Float64s([]float64{2.5, 1.1})</code>
Verifica	<code>IntsAreSorted(s)</code>	Restituisce <code>true</code> se la slice è già ordinata.	<code>sort.IntsAreSorted(s)</code>
	<code>StringsAreSorted(s)</code>	Verifica se le stringhe sono in ordine alfabetico.	<code>sort.StringsAreSorted(s)</code>
Personalizzato	<code>Slice(s, func)</code>	Ordina qualsiasi slice usando una funzione di confronto.	<code>sort.Slice(s, func(i, j int) bool { return s[i] < s[j] })</code>
	<code>SliceStable(s, func)</code>	Come <code>Slice</code> , ma mantiene l'ordine originale tra elementi uguali.	<code>sort.SliceStable(...)</code>
Ricerca	<code>SearchInts(s, x)</code>	Trova l'indice di <code>x</code> in una slice ordinata (ricerca binaria).	<code>sort.SearchInts(ordinato, 10)</code>

```
numeri := []int{5, 2, 9, 1}
sort.Ints(numeri)
fmt.Println(numeri) // [1 2 5 9]
```

```
type Studente struct {
    Nome string
    Media float64
}

studenti := []Studente{
    {"Mario", 24.5},
    {"Anna", 29.0},
    {"Luca", 22.0},
}

// Ordiniamo per Media decrescente
sort.Slice(studenti, func(i, j int) bool {
    return studenti[i].Media > studenti[j].Media
})
```